TIPS: Timing-Independent PMU Phase Signatures for Trace-Driven Multicore Resource Policies

Keshav K.
Hardware Counter Phase Analysis Artifact

***Abstract*—Multicore resource managers need low-overhead signals that indicate when workload behavior changes, but timing-derived PMU metrics such as cycles, IPC, and elapsed time are sensitive to DVFS, thermal, and scheduling effects. TIPS uses instruction-normalized and access-normalized PMU ratios to construct per-core phase signatures and treats phase-change detection as the control signal. On the available PARSEC artifact, TIPS analyzes 110,785 intervals and 58,777 windows from 300 merged runs. Exact next-phase prediction remains weak: the best model reaches 0.367 accuracy. Phase-change detection is stronger: transformer reaches change-F1 0.863, and a compact student tree reaches 0.860. A trace-driven co-scheduling replay shows how these signals can be consumed by a resource policy, but it is not a measured online speedup. Online Burst, DVFS, CAT/resctrl, and prefetch-control measurements are included only when their result files exist.**

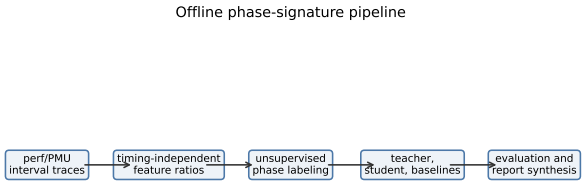

Fig. 1. Artifact pipeline: collection, timing-independent features, labels, predictors, replay, and paper outputs.

I. Introduction

Phase detection matters only if it helps a system make decisions. Prior phase systems established that programs exhibit recurring behavior, but many rely on code signatures, timing metrics, heavy offline models, or do not close the loop with a low-cost multicore policy. The central claim of this draft is deliberately narrow: timing-independent PMU signatures are a defensible substrate for multicore phase-change detection, and the resulting phase-change stream can drive a trace-driven resource-policy replay.

The current evidence does not support a claim of online performance improvement. It supports three narrower claims: phase-change prediction is substantially more reliable than exact next-phase prediction; learned phase IDs behave as reusable resource-behavior classes rather than benchmark names; and a compact detector has plausible storage and operation cost. The missing evidence is an online co-scheduling/cache/prefetch experiment that measures real speedup or fairness.

Contributions. (1) We formalize a timing-independent PMU feature set that excludes cycles, IPC, CPI, elapsed time, per-time rates, and stall-derived metrics. (2) We evaluate phase-change prediction across three multicore PARSEC settings with grouped-by-run splits and no observed run leakage. (3) We quantify phase sharing and workload purity to show that phases are behavioral equivalence classes. (4) We implement a trace-driven resource-policy replay and a reproducible artifact pipeline. (5) We provide ablations, hardware-cost estimates, and explicit pending scripts for online validation.

II. Background and Motivation

SimPoint and BBV work showed that recurring execution behavior can stand in for whole-program simulation [1]–[3]. Runtime phase tracking showed that compact signatures can be used online [4]. Parallel phase work warns that global aggregation can hide asynchronous thread behavior [5], [6]. PMU-based predictors such as P4 and phase-aware multicore forecasting motivate counters and learning, but they do not by themselves establish a cheap resource-control path for this artifact [7], [8].

III. Design

TIPS builds local signatures from branch behavior, L1 access mix, LLC/offcore pressure, and optional uncore bandwidth. Shared context is auxiliary; the local stream remains the unit of classification. The online detector is a centroid table with EWMA smoothing and persistence filtering. A candidate phase change must persist before a resource policy acts, because the policy target is avoiding disruptive changes, not maximizing exact phase-ID accuracy.

IV. Implementation

The implemented artifact includes feature construction, FGMM labeling, classical baselines, transformer and student-tree predictions, detector ablations, validation checks, uniqueness metrics, hardware-cost accounting, and trace-driven policy replay. The replay assigns each phase to a coarse resource class and measures predicted co-runner conflicts over collected windows. It reports proxy weighted-speedup/fairness metrics derived from conflict rate; these are not measured runtime speedups.

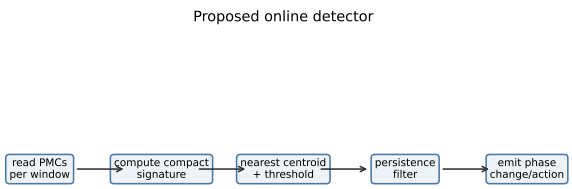

Fig. 2. Online detector structure. The current artifact implements the detector/replay in software; hardware/firmware integration is a cost estimate, not a tapeout.

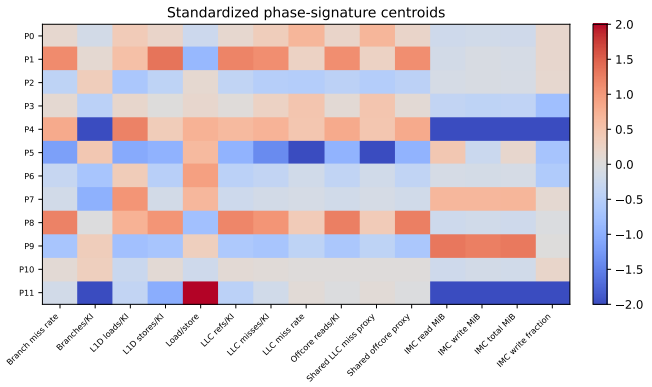

Fig. 3. Phase centroids over timing-independent indicators.

V. Methodology

The available dataset contains 110,785 intervals, 58,777 windows, 15 features, and 300 merged PARSEC runs. The split is grouped by run ID; validation found 0 leaking runs. The evaluation set contains 18398 windows per model. Existing tests pass, and validation outputs are generated under `results/a_star_analysis`.

VI. Results

A. RQ1: Are timing-independent signatures stable?

The current artifact shows within-phase variance reduction and phase centroids over timing-independent features (Fig. 3). A DVFS/timing stress test has not been collected in this artifact instance; the paper must not claim robustness under controlled frequency changes until those result files exist.

B. RQ2: Are phases shared across programs?

The uniqueness analysis finds 10 phases present in at least four workloads and 2 workload-specific phases. Mean dominant-workload share is 0.379. This supports the equivalence-class interpretation and argues against treating phase IDs as benchmark names.

TABLE I
Prediction results. Exact next-phase accuracy is weak; phase-change F1 is the relevant control metric.

Model	Samples	Acc.	Macro-F1	Change-F1
decision_tree	18398	0.341	0.253	0.785
last_value	18398	0.247	0.241	0.000
linear_svm	18398	0.277	0.251	0.763
logistic_regression	18398	0.367	0.321	0.790
nearest_centroid	18398	0.116	0.100	0.812
rle_markov	18398	0.278	0.226	0.679
transformer	18398	0.338	0.292	0.863
student_decision_tree	18398	0.263	0.202	0.860

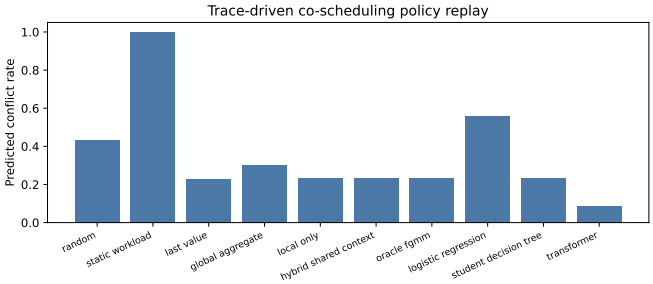

Fig. 4. Trace-driven co-scheduling replay. Lower predicted conflict rate is better. These are proxy replay results, not online speedups.

C. RQ3: How well are phase changes detected?

The best exact next-phase accuracy is only 0.367 from logistic regression. The best phase-change F1 is 0.863 from transformer. The transformer reports 8.555 microseconds per sample in batched GPU inference; the student tree preserves change-F1 0.860 with a compact threshold model.

D. RQ4: Does a phase-aware policy reduce predicted conflicts?

The best replay conflict policy is transformer with conflict rate 0.087. This is sufficient to motivate online experiments, but not sufficient to claim measured performance improvement.

E. RQ5: What is the cost?

F. RQ6: What ablations matter?

The best detector ablation row has phase-change F1 0.812 using feature group branch, metric manhattan, persistence 1, and precision float. The worst transfer setting reaches phase-change F1 0.491, which should be reported as a limitation rather than hidden.

VII. Discussion

Weak exact next-phase accuracy is not a side issue; it prevents claims about exact future phase identity. The defensible control signal is phase change. The current replay suggests how a scheduler could consume the signal, but the paper must not claim real performance benefit until live placement, CAT, prefetch, or bandwidth-throttling experiments are run.

TABLE II
TRACE-DRIVEN POLICY REPLAY. PROXY SPEEDUP/FAIRNESS VALUES ARE MONOTONIC TRANSFORMS OF CONFLICT RATE, NOT MEASURED EXECUTION SPEEDUPS.

Policy	Conflict	Migrations	WS proxy	Fairness
random	0.434	38605	0.698	0.566
static_workload	1.000	0	0.500	0.000
last_value	0.227	2298	0.815	0.773
global_aggregate	0.303	1888	0.768	0.697
local_only	0.232	2348	0.811	0.768
hybrid_shared_context	0.232	2348	0.811	0.768
oracle_fgmm	0.232	2348	0.811	0.768
logistic_regression	0.559	5297	0.642	0.441
student_decision_tree	0.234	16297	0.810	0.766
transformer	0.087	5407	0.920	0.913

TABLE III
DETECTOR COST ESTIMATE FOR 8 FEATURES AND 16 CENTROIDS PER CORE.

Bits	Bytes/core	Ops/window
8	160	128
10	194	128
12	228	128
16	296	128

VIII. RELATED WORK

The closest work spans BBV/SimPoint phase analysis [1]–[3], runtime phase tracking [4], parallel phases [5], [6], PMU-based prediction [7], [8], and contention-aware resource management [9]–[11]. TIPS is weaker than these papers on online validation today, but its intended niche is timing-independent PMU phase-change detection plus low-cost resource-policy integration.

IX. LIMITATIONS

If online co-scheduling or DVFS result files are absent, the corresponding claims are omitted from the results and retained as limitations. Intel CAT/resctrl is unavailable locally unless the target node exposes `/sys/fs/resctrl`. Prefetch controls require platform-specific support and are not claimed unless measured. PMU multiplexing and limited counter slots remain risks. PARSEC-only evaluation limits generality.

X. CONCLUSION

Timing-independent PMU signatures are promising for multicore phase-change detection, but the current artifact should be presented as a validated detection and trace-replay study, not as a completed resource-management system. The next decisive experiment is live phase-aware placement or cache/bandwidth control with measured speedup and fairness.

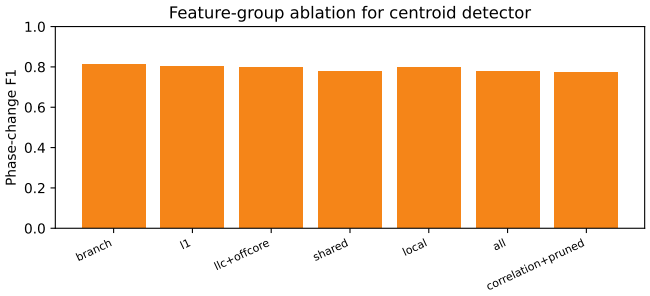

Fig. 5. Feature-group ablation for the centroid detector.

TABLE IV
PMU SLOT SENSITIVITY.

Slots	Features	Acc.	Change-F1
2	2	0.113	0.810
4	4	0.158	0.786
6	6	0.158	0.794
8	8	0.156	0.798

REFERENCES

[1] T. Sherwood, E. Perelman, and B. Calder, "Basic block distribution analysis to find periodic behavior and simulation points in applications," in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, 2001, pp. 3–14.

[2] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, "Automatically characterizing large scale program behavior," in *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, 2002, pp. 45–57.

[3] G. Hamerly, E. Perelman, J. Lau, and B. Calder, "Simpoint 3.0: Faster and more flexible program phase analysis," *Journal of Instruction-Level Parallelism*, vol. 7, pp. 1–28, 2005.

[4] T. Sherwood, S. Sair, and B. Calder, "Phase tracking and prediction," in *Proceedings of the 30th International Symposium on Computer Architecture*, 2003, pp. 336–347.

[5] E. Perelman, M. Polito, J.-Y. Bouguet, J. Sampson, B. Calder, and C. Dulong, "Detecting phases in parallel applications on shared memory architectures," in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, 2006.

[6] C.-H. Chang, P. Liu, and J.-J. Wu, "Sampling-based phase classification and prediction for multi-threaded program execution on multi-core architectures," in *Proceedings of the International Conference on Parallel Processing*, 2013.

[7] Y. Kim, P. Mercati, A. More, E. Shriver, and T. Rosing, "P4: Phase-based power/performance prediction of heterogeneous systems via neural networks," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2017, pp. 683–690.

[8] E. S. Alcorta and A. Gerstlauer, "Learning-based phase-aware multi-core cpu workload forecasting," *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 2, pp. 23:1–23:27, 2023.

[9] A. Fedorova, S. Blagodurov, and S. Zhuravlev, "Managing contention for shared resources on multicore processors," *ACM Queue*, vol. 8, no. 1, 2010.

[10] L. Subramanian, V. Seshadri, Y. Kim, B. Jaiyen, and O. Mutlu, "MISE: Providing performance predictability and improving fairness in shared main memory systems," in *Proceedings of the 19th International Symposium on High Performance Computer Architecture*, 2013.

[11] A. Navarro-Torres, J. Alastruey-Benede, P. Ibanez, and V. Vinals-Yufera, "Balancer: Bandwidth allocation and cache partitioning for multicore processors," *The Journal of Supercomputing*, vol. 79, pp. 10 252–10 276, 2023.

[12] G. Hamerly, E. Perelman, J. Lau, B. Calder, and T. Sherwood, "Using machine learning to guide architecture simulation," *Journal of Machine Learning Research*, vol. 7, pp. 343–378, 2006.

[13] C. Ding, S. Dwarkadas, M. C. Huang, K. Shen, and J. B. Carter, "Program phase detection and exploitation," in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, 2006.

[14] I. E. Ziedan, H. I. Shehata, and S. M. Serag, "A run-time program phase detection technique for optimizing per-phase l2 cache demand," *The*

Egyptian International Journal of Engineering Sciences and Technology, 2016.

[15] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: Characterization and architectural implications," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*, 2008, pp. 72–81.

[16] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *Proceedings of the 13th International Symposium on High Performance Computer Architecture*, 2007, pp. 63–74.

[17] L. Carpentieri, A. D. Caro, M. S. Beni, K. Fan, and B. Cosenza, "Phase-based frequency scaling for energy-efficient heterogeneous computing," in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, 2025.

[18] J. Nomani and J. Szefer, "Predicting program phases and defending against side-channel attacks using hardware performance counters," in *Proceedings of the Workshop on Hardware and Architectural Support for Security and Privacy*, 2015.

[19] W. Zhang, J. Li, Y. Li, and H. Chen, "Multilevel phase analysis," *ACM Transactions on Embedded Computing Systems*, vol. 13, no. 4s, 2014.

[20] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B*, vol. 58, no. 1, pp. 267–288, 1996.

[21] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1967, pp. 281–297.

[22] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the em algorithm," *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–38, 1977.

[23] C. Spearman, "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.

[24] I. T. Jolliffe, *Principal Component Analysis*. Springer, 2002.

[25] A. K. Jain, "Data clustering: 50 years beyond k-means," *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010.

[26] L. Breiman, J. Friedman, R. Olshen, and C. Stone, "Classification and regression trees," *Wadsworth Statistics/Probability Series*, 1984.

[27] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.

[28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[29] S. Zhuravlev, S. Blagodurov, and A. Fedorova, "Addressing shared resource contention in multicore processors via scheduling," in *Proceedings of the Fifteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2010, pp. 129–142.

[30] J. Mars, L. Tang, R. Hundt, K. Skadron, and M. L. Soffa, "Bubble-up: Increasing utilization in modern warehouse scale computers via sensible co-locations," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, 2011, pp. 248–259.

[31] M. K. Qureshi and Y. N. Patt, "Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches," in *Proceedings of the 39th Annual IEEE/ACM International Symposium on Microarchitecture*, 2006, pp. 423–432.

[32] A. S. Dhodapkar and J. E. Smith, "Managing multi-configuration hardware via dynamic working set analysis," in *Proceedings of the 29th Annual International Symposium on Computer Architecture*, 2002, pp. 233–244.

[33] R. Balasubramonian, D. Albonesi, A. Buyuktosunoglu, and S. Dwarkadas, "Memory hierarchy reconfiguration for energy and performance in general-purpose processor architectures," in *Proceedings of the 33rd Annual IEEE/ACM International Symposium on Microarchitecture*, 2000, pp. 245–257.

[34] V. M. Weaver, "Linux perf event features and overhead," *The Second International Workshop on Performance Analysis of Workload Optimized Systems*, 2013.

[35] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2024.

[36] Advanced Micro Devices, *Processor Programming Reference for AMD Family Processors*, 2024.

[37] Arm Ltd., *Arm Architecture Reference Manual: Performance Monitors Extension*, 2024.